

ASTERIA: Technical Report

AI-Assisted Virtual Camera Tracking System for Free-Space Optical Communication Coarse Alignment

Repository: [soham-salunkhe/Asteria](https://github.com/soham-salunkhe/Asteria)

Branch reviewed: main

Commit reviewed: 1a53503 (feat(benchmark): honest lock-retention diagnostics, dominant-flux detection, turnaround feedforward)

Report author: Manus AI

Assessment date: 17 September 2026

Report scope: Repository and implementation analysis, test/build verification, and engineering recommendations

Executive Summary

ASTERIA is a software simulation of the coarse Pointing, Acquisition and Tracking (PAT) stage used in free-space optical communication (FSOC). Its purpose is to demonstrate how a mobile optical terminal can detect a beacon, estimate its image-plane motion, steer a pan/tilt camera, and maintain alignment under target motion, sensor noise, atmospheric degradation, platform motion, and camera disturbances. The repository implements this as a full-stack application rather than as an isolated algorithm: a Python simulation and tracking service provides the control loop, a React and TypeScript client presents mission telemetry, a Node.js service proxies an engineering copilot, and an Electron shell packages the desktop experience.

The central technical contribution is the closed-loop pipeline. A virtual target is advanced in a three-dimensional world. The camera projects the target into a narrow field of view. A rendered sensor frame is degraded by configurable disturbances. A vision detector extracts candidate bright regions without using ground-truth coordinates. A Kalman filter estimates centroid position and velocity. Pixel error is converted to angular error. A dual-axis PID controller commands pan and tilt corrections. The engine updates state, records metrics, and streams telemetry over WebSocket. This arrangement closely matches the causal structure of a real tracking system and makes the simulator useful for algorithm development, demonstrations, and repeatable acceptance testing.

The repository is technically stronger than its age and presentation might suggest. The implementation includes real numerical filtering, control, projection, candidate-based image detection, SQLite telemetry persistence, CSV/JSON/PDF reporting, video-input processing, and a 39-test Python validation suite. Independent verification on the reviewed checkout produced **39 passing tests**, a successful frontend production build, a successful backend TypeScript build, and a successful Electron TypeScript build. The frontend build does emit a warning because two minified JavaScript chunks exceed 500 kB; this is a performance and maintainability concern rather than a build failure.

The main limitations are equally important. The default detector is a classical image detector rather than a trained deep-learning detector. The YOLO abstraction is integration-ready, but a trained model and labeled optical-beacon dataset are not included. External-video metrics lack true ground truth, so their tracking error is necessarily a proxy. The test suite is strong at unit and component acceptance behavior but does not provide broad end-to-end service tests, load tests, hardware-in-the-loop validation, or statistical confidence intervals across randomized scenarios. The documentation also contains a stale `PROJECT_ANALYSIS.md` describing an unrelated tourism platform; it should not be treated as evidence about ASTERIA.

The recommended next stage is not a wholesale rewrite. ASTERIA should preserve its modular pipeline and invest in reproducible benchmark datasets, statistical experiment orchestration, end-to-end WebSocket tests, detector/model evaluation, stricter deployment security, frontend code-splitting, and explicit configuration/version provenance. These improvements would convert a compelling simulation and demonstration platform into a defensible engineering evaluation tool.

1. Problem Understanding and Scope

1.1 Operational problem

FSOC terminals transmit information through a narrow optical beam. Before data transfer can begin, the transmitting and receiving terminals must point their optical axes toward one another. The coarse PAT stage narrows the initial pointing uncertainty until a sufficiently stable alignment is achieved. In a real deployment, the problem is difficult because the beacon may move, the host platform may vibrate or translate, the image may be noisy or degraded by atmosphere, and the camera can only observe a limited field of view at a time.

ASTERIA addresses the software side of this problem. It does not claim to replace an optical terminal, gimbal, detector, or physical test range. Instead, it provides a controllable virtual environment in which the full feedback loop can be inspected. This separation is valuable because it allows tracking logic to be evaluated before expensive hardware is available, while still exposing the engineering trade-offs that matter in a real system: field of view, slew limits, detector confidence, filter convergence, control stability, reacquisition, and lock retention.

1.2 Functional objective

The system must repeatedly answer five questions during a run:

1. **Where is the beacon in the camera image?**
2. **Where is the beacon likely to be on the next frame?**
3. **How far is the optical axis from the target direction?**
4. **What pan and tilt motion will reduce that error without violating gimbal limits?**
5. **Has alignment been stable long enough to declare lock, and can it be recovered after loss?**

The implementation exposes these answers through a state machine, telemetry packets, visual overlays, and stored run reports. The key design choice is that the detector consumes rendered image data. It is not allowed to use the simulator's true target pixel position as its measurement. Ground truth remains available for evaluation in synthetic mode, but it is kept separate from the observation path.

1.3 Scope and non-goals

The reviewed code supports synthetic targets, multiple motion patterns, disturbance injection, video-file processing, single and multiple registered entities, live telemetry, and report export. It is not a complete FSOC communications stack. It does not simulate modulation, coding, link budgets, photodiode acquisition, atmospheric channel attenuation with calibrated physical units, or hardware gimbal interfaces. It also does not establish that the selected controller and detector will meet a particular fielded terminal's requirements without hardware-specific calibration.

1.4 Assessment boundaries

This report analyzes the main branch at commit 1a53503. Claims are categorized implicitly as follows:

- **Verified:** directly observed in source or reproduced by a command during this review.
- **Documented:** stated in repository documentation but not independently reproduced in this review.
- **Recommended:** an engineering improvement derived from observed risks or gaps.

This distinction matters because the repository contains both current implementation documents and a stale analysis file from an unrelated project. The executable FSOC code, FSOC README, requirements analysis, tests, and current commit history were treated as authoritative.

2. System Architecture

2.1 Logical architecture

ASTERIA uses a service-oriented architecture with a simulation engine at the center. The frontend is responsible for operator interaction and visualization. The Python service owns simulation truth, image formation, detection, estimation, control, metrics, persistence, and report generation. The Node service provides a separate HTTP endpoint for the Gemini engineering copilot. Electron starts and supervises the local services in the desktop distribution.

```

flowchart LR
    UI[React + TypeScript UI\nMission Control and telemetry pages]
    WS[WebSocket\ntelemetry stream]
    API[FastAPI\nREST control and reports]
    ENG[Simulation Engine\n30 Hz closed loop]
    ENV[Target, camera, environment\nand disturbances]
    IMG[Frame renderer\nsynthetic sensor image]
    DET[Beacon detector\nimage candidates / YOLO-ready]
    KF[2-D Kalman filter]
    PID[Dual-axis PID]
    DB[(SQLite\nscenarios, runs, telemetry)]
    REP[CSV / JSON / PDF reports]
    COP[Node/Express\nGemini copilot proxy]
    DESK[Electron desktop shell]

    UI --> API
    UI <--> WS
    API --> ENG
    ENG --> ENV --> IMG --> DET --> KF --> PID --> ENV
    ENG --> DB
    DB --> REP --> UI
    UI --> COP
    DESK --> API
    DESK --> COP

```

2.2 Runtime responsibilities

The Python service is launched with Uvicorn and exposes REST endpoints under `/api`. `fsoc_main.py` creates the FastAPI application, initializes the SQLite database, configures CORS, registers the simulation engine and video processor, and manages WebSocket subscribers. The connection manager retains the latest telemetry packet, which allows a client that subscribes after processing has begun to receive a current frame rather than remaining on an empty canvas.

The `SimulationEngine` runs the frame loop in a background `asyncio` task. The documented loop order is meaningful: target update precedes camera projection; image generation precedes detection; detection precedes Kalman update and prediction; error calculation precedes PID control; metrics and telemetry are generated after the control action. SQLite writes are moved to a single-worker thread pool so database I/O does not block the event loop.

The frontend is a Vite-built React application. `useSimulation` and the WebSocket service provide the principal state boundary between the UI and the tracking service. Pages are organized around mission control, live tracking, detection, camera control, disturbances, analytics, reports, settings, and the copilot. The visualization layer includes a synthetic camera feed, a 3-D scene, telemetry panels, state badges, event logs, and charts.

The Electron layer is a packaging and orchestration boundary. It discovers available local ports, starts the tracking engine and copilot service, creates a hardened `BrowserWindow` with context isolation and no Node integration, and exposes a small preload bridge for configuration, video-file selection, report saving, and window controls. This is a sensible separation: the browser UI does not receive unrestricted filesystem or process access.

2.3 Data flow

A synthetic frame begins with a target position in world coordinates. The virtual camera transforms the relative target vector into camera coordinates, checks whether the target is in front of the aperture and inside the horizontal and vertical field of view, and maps angular offsets to pixels. The frame renderer then creates a float image and applies selected image noise and atmospheric effects. The detector processes that image and returns a `DetectionResult` containing class, confidence, bounding box, centroid, inference time, and candidate diagnostics.

The control path uses detector output rather than simulator truth. When a detection is present, the Kalman filter updates its state and then predicts forward. The engine converts pixel error to gimbal error using the project's authoritative camera constants. The PID controller produces bounded pan and tilt commands. The camera applies those commands subject to its angular limits. The resulting telemetry includes both measured observation data and simulator-only diagnostic data, which enables honest evaluation without contaminating the control loop.

3. Software Modules

3.1 Simulation and world model

The simulation package contains the environment, target, camera, and disturbance modules. A target uses a right-handed coordinate convention: X is East, Y is Up, and Z is North toward the camera. The target model supports linear, sinusoidal, circular, figure-eight, spiral, random-walk, and static trajectories. Parametric trajectories calculate their instantaneous derivatives so the velocity state is consistent with position rather than being an unrelated display value.

The camera model represents a pan/tilt gimbal. Its default runtime constants are a 640×480 image, a narrow 4° horizontal field of view, a 3° vertical field of view, and a 30 Hz frame rate. Pan is constrained to $\pm 180^\circ$, while tilt is constrained to $\pm 89^\circ$ to avoid a singularity at $\pm 90^\circ$. The projection implementation explicitly handles the camera-space forward direction; therefore a target can be found after the gimbal rotates to a different hemisphere. This is more robust than assuming that the target always lies in the initial +Z direction.

The disturbance engine separates state perturbations from observation degradation. Atmospheric turbulence and platform vibration create angular perturbations. Camera motion adds random angular disturbance. Sensor noise is applied to the rendered image, not to the ground-truth state. Target-motion variation changes target velocity. Configuration inputs are sanitized and finite-value checks prevent malformed REST payloads from injecting NaN or infinity into the simulation loop.

3.2 Frame renderer

`vision/frame_renderer.py` produces the sensor image consumed by the detector. It renders a beacon with configurable intensity, shape, and size, then applies noise modes such as Gaussian, salt-and-pepper, and Poisson noise. Atmospheric modes such as haze, fog, rain, and low light alter the detection image rather than merely changing a frontend overlay. This distinction is technically important: if atmosphere only changed the screen decoration, detector performance would not be measured honestly.

The renderer also supports deterministic seeds in tests. Determinism is not the default operating condition for a realistic simulation, but it is essential for regression tests and reproducible benchmark cases.

3.3 Vision detection

The current default detector is `ImageBeaconDetector`. It uses OpenCV connected-component analysis and a candidate-scoring pipeline. The detector first denoises with a median filter, computes image statistics, selects an adaptive threshold, and generates connected components. It includes fallbacks for flat bright backgrounds and low candidate counts. Candidate features include peak intensity, mean brightness, local contrast, compactness, fill ratio, aspect ratio, expected area, and temporal or predicted-position consistency.

The detector also includes safeguards against common false positives. Small border artifacts are rejected. Long glare streaks are re-thresholded to isolate a compact bright core. A nearest-neighbor association radius discourages frame-to-frame jumps to unrelated stars or noise blobs. The result contains all candidates for UI diagnostics, not only the selected candidate. This is an effective design for a simulator because it makes failure analysis visible.

The architecture includes a detector interface and a YOLO-ready path. However, “YOLO-ready” means that the integration seam exists; it does not mean that a trained YOLO model has been evaluated. A fielded system would need labeled beacon imagery, domain-specific augmentation, model versioning, inference acceleration, confidence calibration, and a comparison against the classical detector.

3.4 State estimation

`KalmanFilter2D` implements a linear constant-velocity filter with state vector $[x, y, v_x, v_y]$. The measurement vector is the detected centroid $[x, y]$. The prediction matrix uses the measured frame interval, and the process covariance is formed from the conventional constant-acceleration noise model. On the first detection the filter initializes position and zero velocity. Later detections update the state through innovation covariance and Kalman gain calculations.

The filter exposes position, velocity, covariance trace, convergence status, predicted position, and reset behavior. The covariance trace is used as a compact uncertainty indicator. This is appropriate for telemetry, although a more complete evaluation should report axis-specific covariance and normalized innovation statistics.

3.5 Control and state machine

The PID controller is dual-axis. It computes independent corrections for pan and tilt, applies proportional, integral, and derivative terms, filters the derivative contribution, clamps the output to a maximum angular velocity, and uses anti-windup logic. The camera then integrates the commanded rate over the frame interval and clamps the absolute gimbal angles.

The engine's target states are READY, SEARCHING, DETECTED, ACQUIRING, TRACKING, LOCKED, LOST, REACQUIRING, and ERROR. The search behavior uses an expanding and deterministic sweep rather than an omniscient look-at operation. During tracking, a coast period allows the Kalman estimate to bridge short detector dropouts. A longer loss transitions to a search or reacquisition behavior. Lock is not a single-frame event: the implementation requires stable frames and a pixel-error threshold.

The state machine is one of the strongest parts of the repository because it expresses operational behavior rather than only numerical error minimization. A realistic tracker must specify what happens when the beacon is first seen, temporarily lost, fully lost, reacquired, or manually manipulated.

3.6 Metrics and persistence

RunMetrics accumulates acquisition time, reacquisition time, detection confidence, processing time, FPS, angular error, pixel error, lock retention, target losses, and event information. The SQLite layer stores scenarios, simulation runs, and telemetry samples. Synthetic simulation samples are intentionally downsampled for database persistence so that the database does not become the frame-loop bottleneck. Video processing can persist complete telemetry after processing so benchmark reports remain reproducible.

The report package exports CSV, JSON, and PDF. This is useful for both human review and downstream analysis. A future schema should add explicit units and provenance fields to every metric, especially for the distinction between synthetic ground-truth error and video-mode proxy error.

3.7 Frontend and desktop modules

The React UI provides operational views rather than a single dashboard. Mission Control starts and configures a run. Live Tracking displays the camera view and current telemetry. Detection exposes the computer-vision pipeline. Camera Control allows manual pan/tilt and PID tuning. Disturbances exposes environmental and platform effects. Analytics charts historical telemetry. Reports lists run history and exports. Settings exposes parameters. Copilot provides AI-assisted engineering commentary.

The frontend also includes 2-D and 3-D representations, state-based color coding, event logs, and visual indicators for lock quality. The design is appropriate for a demonstration system because it shows the causal chain from scene to detector to controller rather than hiding everything behind a single score.

4. Tracking Methods and Control Mathematics

4.1 Projection and measurement geometry

Let the target position relative to the camera be (dx, dy, dz) . The camera first rotates this vector by the inverse pan/tilt transformation to obtain camera-frame coordinates (x_2, y_2, z_2) . If $z_2 \leq 0$, the target is behind the optical aperture. Otherwise, horizontal and vertical angular offsets are:

$$[\alpha_h = \tan^{-1}(x_2/z_2), \quad \alpha_v = \tan^{-1}(y_2/z_2).]$$

For horizontal and vertical fields of view F_h and F_v , the pixel coordinates are:

$$[p_x = \left(\frac{\alpha_h}{F_h/2} + 1\right) \frac{W}{2}, \quad p_y = \left(1 - \frac{\alpha_v}{F_v/2} + 1\right) \frac{H}{2}.]$$

The implementation preserves the image convention that Y increases downward, while positive tilt means upward optical motion. Tests explicitly verify the sign inversion at this boundary. This is a common source of control instability, so the presence of dedicated direction tests is significant.

4.2 Detection and filtering sequence

The observation sequence is:

1. Render the current world state into a camera image.
2. Apply image noise and atmosphere.
3. Extract bright connected components.

4. Score and select a candidate.
5. Update the Kalman filter with the selected centroid.
6. Predict the next centroid and expose uncertainty.
7. Compute control error.

The detector does not receive the simulator's true pixel coordinates as an input. The predicted position may influence candidate association, but it is not a replacement measurement. This keeps the closed loop causal and allows the system to demonstrate failure under occlusion, noise, or distractors.

4.3 Pixel-to-gimbal error

The tracking controller converts image error into angular error. Let the image center be (c_x, c_y) and the measured centroid be (p_x, p_y) . The image errors are:

$$[e_x = p_x - c_x, \quad e_y = p_y - c_y.]$$

With the project's camera constants, pixel error is mapped to pan and tilt error. The vertical mapping reverses sign because a smaller image Y coordinate indicates a beacon above the optical axis. Tests cover left, right, above, and below cases and verify that the resulting gimbal command moves in the expected direction.

4.4 PID control

For each axis, the controller uses:

$$[u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}.]$$

The implementation is discrete and rate-limited. The integral term is bounded to prevent windup when the gimbal is saturated. The derivative term is filtered to reduce amplification of detector noise. The command is integrated over dt and clamped to the physical pan/tilt range. This is preferable to teleporting the camera to the target direction because it exposes settling time, overshoot, saturation, and disturbance rejection.

4.5 Lock, loss, and reacquisition

Lock is declared only after the detector and controller remain within the configured pixel threshold for the required number of frames. Temporary detector misses are bridged by a coast period. Prolonged misses transition to LOST; reacquisition is tracked separately and contributes to reacquisition count, average time, maximum time, and lock-retention statistics.

This distinction avoids a misleading metric in which a tracker that repeatedly drops and instantly relocks appears equivalent to a tracker that never loses the target. The current branch's benchmark-oriented commit further emphasizes honest lock-retention diagnostics and dominant-flux detection.

5. AI and Intelligent Methods

5.1 What is AI in ASTERIA?

ASTERIA is AI-assisted in two different senses. First, it has an intelligent perception and estimation pipeline: image-based candidate detection, Kalman state estimation, and adaptive control. Second, it optionally exposes a Gemini-based engineering copilot through the Node service. The core closed-loop tracker does not currently depend on a trained neural network to function.

This distinction should be stated clearly in project submissions. Calling the system an "AI tracking system" is defensible if AI is used broadly to describe automated perception and estimation, but the repository does not provide evidence that the default run is powered by a trained deep detector. The current default detector is classical computer vision, while the YOLO detector is an extension point.

5.2 Classical vision detector

The image detector uses adaptive thresholding, connected components, feature scoring, spatial association, and confidence estimation. It is explainable: an operator can inspect candidate bounding boxes, score components, confidence, and rejection behavior. This is particularly suitable for a research simulator because failure modes are easier to diagnose than opaque model decisions.

Its limitations are exposure sensitivity, handcrafted feature dependence, and possible degradation when the beacon's appearance differs substantially from the renderer's assumptions. Candidate scoring may also require tuning across camera resolution, atmospheric mode, blur, compression, and distractor distributions.

5.3 Kalman filtering as model-based intelligence

The Kalman filter is not a machine-learning model, but it is a model-based inference method. It combines a motion prior with noisy centroid observations. Its strengths are low latency, small computational cost, interpretable uncertainty, and graceful short-term prediction through missed frames. Its constant-velocity assumption is less accurate during abrupt accelerations, circular motion at high curvature, or rapidly changing platform motion. A constant-acceleration or interacting-multiple-model filter could be evaluated as a future alternative.

5.4 Gemini engineering copilot

The Node/Express backend includes a guide route that can pass engineering questions and live telemetry context to Gemini. This feature is advisory; it should not be placed in the hard real-time control loop. AI-generated explanations should be clearly labeled as analysis rather than authoritative control commands. Input validation, rate limiting, secret handling, and telemetry redaction are necessary if the copilot is deployed beyond a local demonstration.

5.5 YOLO integration path

The repository's detector abstraction supports a drop-in YOLO implementation. To make that path an evaluated AI method rather than a design promise, the project needs a dataset specification, train/validation/test splits, annotation rules, model configuration, training procedure, precision/recall metrics, false-positive analysis, inference-latency measurement, and a comparison against ImageBeaconDetector. Model weights should be versioned by checksum and paired with the renderer or real-video domain used for evaluation.

6. Interfaces, Configuration, and Data Management

6.1 REST API

The FastAPI layer supports simulation lifecycle commands such as start, stop, pause, reset, demo termination, and entity clearing. It also supports live PID, Kalman, disturbance, atmosphere, camera, target, scenario, run, report, and video-upload operations. Pydantic request models constrain many numeric values, including pan/tilt bounds, PID gains, Kalman parameters, and atmospheric strength.

The entity registry supports targets, cameras, satellites, and target-to-camera relationships. This is a useful foundation for multi-terminal experiments, although the current control loop remains centered on one active tracking session.

6.2 WebSocket telemetry

The WebSocket endpoint streams simulation telemetry to the frontend. Packets contain camera state, target state, detection information, Kalman state, PID output, disturbances, metrics, and event information. Retaining the latest message helps late subscribers, but the approach is still a latest-state stream rather than a durable event log. Consumers that require every frame should use persisted telemetry or a sequence-aware transport.

6.3 Storage model

SQLite is a reasonable choice for a local simulator and desktop application. It minimizes deployment complexity and supports scenario/run history and report export. The database design separates run summaries from telemetry samples. The single-worker executor reduces contention with the asyncio loop.

For multi-user or distributed deployment, SQLite would eventually need to be replaced or isolated behind a database service. Before that transition, the schema should define indexes, retention policy, schema version, units, and experiment provenance. Every run should record the commit, configuration hash, random seed, detector name, model version, and environment metadata.

6.4 Desktop packaging

Electron enables native file dialogs and a self-contained operator experience. The preload bridge exposes only selected functions and the window disables Node integration. The main process dynamically resolves ports, which helps avoid collisions. The deployment model is nevertheless Windows-oriented in places, including process termination through `taskkill`; cross-platform process supervision should be tested if Linux or macOS packaging is a goal.

7. Test Methodology

7.1 Test organization

The Python test suite is divided into an acceptance suite and a PS169 compliance suite. The tests cover control direction, convergence, state-reset independence, target trajectories, image noise, atmospheric degradation, camera jitter, platform motion, detector behavior, Kalman behavior, metrics, and other acceptance conditions. Fixtures use deterministic renderer seeds where repeatability is needed.

The test structure follows a useful pyramid at the algorithm level. Small tests validate sign conventions and equations. Component tests validate detector, filter, controller, renderer, and disturbance behavior. Acceptance tests exercise combinations of those components. However, the repository contains fewer tests at the service and user-interface levels than at the numerical component level.

7.2 Verified execution

The following checks were run on the reviewed checkout:

Check	Result	Interpretation
<code>python3 -m pytest tests -q</code>	39 passed	Python algorithm and acceptance tests pass.
<code>npm run build:frontend</code>	Success	TypeScript and Vite production build pass.
<code>npm --prefix backend run build</code>	Success	Node backend compiles.
<code>npm run build:electron</code>	Success	Electron main/preload TypeScript compiles.

The frontend build reported two chunks larger than 500 kB after minification. The warning recommends dynamic imports or code-splitting. It does not invalidate the build, but it can affect initial load time and memory use.

7.3 Strengths of the test method

The tests directly inspect physical and control invariants rather than only checking that functions return values. Examples include verifying that a left-side beacon produces a negative pan command, that fixed-image coordinates converge under PID control, that circular paths close after one period, that noise remains within expected bounds, and that atmospheric modes change the detection image. The detector tests use rendered frames and do not need hardware.

The suite also tests reset independence for video sessions, which is important because stale association memory can bias the first frames of a new benchmark. The use of deterministic seeds makes failures reproducible.

7.4 Missing test dimensions

The project would benefit from the following additions:

- End-to-end FastAPI tests that start and stop simulations, connect WebSockets, and verify packet schemas.
- Contract tests between frontend services and the FastAPI routes.
- Browser-level tests for mission configuration, manual camera movement, report download, and video upload.
- Property-based tests for finite outputs, gimbal bounds, and random configurations.
- Load tests for concurrent WebSocket subscribers and long-duration telemetry storage.
- Statistical tests across many seeds, with confidence intervals for acquisition time, lock retention, and error.
- Video benchmark tests using a versioned corpus and annotated ground truth where possible.
- Security tests for CORS behavior, malformed payloads, upload limits, and copilot secret exposure.

8. Performance Analysis

8.1 Verified performance evidence

The strongest directly verified performance result in this review is test and build execution, not a long-running runtime benchmark. The repository's current documentation reports approximately 28–30 FPS in the simulator, 2–8 ms per frame with the mock detector, and low WebSocket latency, but these values were not independently reproduced here. They should therefore be described as documented project measurements rather than as new benchmark results.

The test suite completed in under one second in the review environment, which confirms that the algorithmic components are lightweight enough for rapid regression testing. The frontend production bundle built in approximately one second after dependencies were installed. These values are not substitutes for runtime tracking performance, but they establish that the development feedback loop is fast.

8.2 Throughput and latency considerations

The target frame rate is 30 Hz, corresponding to approximately 33.3 ms per frame. The frame loop performs rendering, image processing, connected components, candidate scoring, Kalman operations, PID control, metric accumulation, telemetry serialization, and occasional persistence. The SQLite executor is separated from the event loop, which protects control latency from synchronous database writes.

The image detector is the most likely CPU cost in synthetic mode, while OpenCV processing and video decoding will dominate external-video mode. The frontend's large React Three Fiber and application bundles affect startup and initial interaction rather than the closed-loop Python timing. The build warning indicates a clear optimization opportunity: lazy-load 3-D visualization and rarely used pages.

8.3 Measurement quality

The metrics design is broad, but performance claims need stronger experimental discipline. A single average FPS can hide frame-time spikes. The system should record p50, p90, p95, p99, maximum frame time, detector time, renderer time, serialization time, and persistence time separately. It should also distinguish simulated time from wall-clock time and report dropped or delayed frames.

Tracking quality should be reported as distributions rather than only averages. Recommended measures include acquisition-time CDFs, angular-error RMSE and percentile bands, lock-retention confidence intervals, reacquisition-time distributions, detection precision/recall, and failure counts by disturbance mode. Benchmark runs should use fixed seeds and configuration hashes so changes can be compared fairly.

8.4 External video caveat

For uploaded videos, the system cannot know the true beacon position unless the video includes annotations or a calibrated reference. Consequently, true angular tracking error is unavailable. The repository documents this limitation and uses detected or image-derived quantities instead. This is the correct conceptual treatment; the report UI must continue to label those quantities as proxies and must not present them as ground-truth error.

9. Limitations, Risks, and Technical Debt

9.1 Model and validation risk

The classical detector is explainable and fast, but its performance envelope is tied to the renderer and selected candidate features. A detector that performs well on synthetic images may fail on lens flare, compression artifacts, sensor bloom, background stars, or real atmospheric scattering. The absence of a labeled real-video corpus prevents a strong claim about field performance.

The Kalman filter assumes constant velocity. The target library contains nonlinear trajectories and random walks, so the estimator is intentionally approximate. This is acceptable for a baseline, but the report should avoid implying that the filter is an optimal estimator for every trajectory.

9.2 Documentation drift

PROJECT_ANALYSIS.md describes YatraAI, a tourism intelligence platform with React, PostgreSQL, and NLP modules. That content conflicts with the repository's FSOC implementation and appears to be stale or accidentally copied. It is a material documentation defect because a reviewer could infer the wrong problem statement, architecture, database, and AI methods. The file should be removed, replaced, or clearly archived.

Some other documentation claims such as "production-quality" or "all requirements complete" are stronger than the evidence available from a source review. A technical submission should use measured language and attach reproducible benchmark artifacts.

9.3 Deployment and security risk

The FastAPI application currently allows all CORS origins and credentials in the reviewed code. That is convenient for local development but unsafe as a default for a networked deployment. Upload endpoints should enforce file size, type, duration, and decoding limits. The copilot service must keep its Gemini key server-side and should use strict request validation and rate limits. The repository includes rate-limit and validation dependencies

in the backend, but deployment configuration should be checked to ensure those controls are active on every relevant route.

9.4 Frontend maintainability risk

The frontend build warning about large chunks suggests that the initial bundle includes more than the first screen requires. Route-level lazy loading, dynamic loading of 3-D components, and a clear bundle budget would improve startup behavior. The project also has both browser and Electron runtime paths; configuration precedence should be tested so that dynamic desktop ports cannot be overwritten by static defaults.

9.5 Persistence and reproducibility risk

Randomness is used in target motion, disturbances, and phase initialization. Some tests seed components, but a complete run-level seed and configuration manifest should be persisted with every run. Without this, two visually similar runs may not be directly comparable. A run should also store the exact git commit, detector implementation, constants profile, and dependency lockfile hash.

10. Future Improvements and Development Roadmap

10.1 Immediate improvements

The first release-hardening step should be documentation cleanup. Replace the stale project analysis, document the actual API and WebSocket schemas, define metric units and ground-truth status, and add a single reproducible benchmark command. The README should link to the architecture diagram, test instructions, and known limitations.

The second step should be measurement instrumentation. Add per-stage timings and frame-time percentiles. Persist the random seed and full configuration. Generate a benchmark summary that compares a baseline commit with the current commit. This would make the existing metrics substantially more useful.

The third step should be deployment hardening. Restrict CORS to configured origins, enforce upload limits, validate route payloads consistently, avoid exposing development defaults in production, and verify copilot rate limiting and error handling.

10.2 Medium-term improvements

Build a benchmark harness that runs a matrix of target trajectories, FOVs, PID gains, detector modes, and disturbance levels. Use fixed seeds and multiple repetitions. Report acquisition success rate, median and tail acquisition time, average and percentile error, lock retention, reacquisition time, frame-time percentiles, and detector precision/recall where labels exist.

Add end-to-end tests that launch the FastAPI service, start a run, consume telemetry, pause and resume the engine, upload a short video, and retrieve a report. Add frontend smoke tests for the most important operator workflows. These tests will catch integration failures that algorithm-level tests cannot see.

Improve estimator flexibility by evaluating a constant-acceleration Kalman model and an interacting-multiple-model filter. The project can select the estimator based on target dynamics, or compare estimators offline before changing the runtime default.

10.3 Long-term improvements

Create a real optical-beacon dataset with diverse backgrounds, atmospheric conditions, motion blur, exposure levels, and distractors. Train and evaluate a lightweight detector such as YOLO in parallel with the classical detector. Measure not only mean average precision but also end-to-end control impact: a detector with higher image precision is not automatically better if it adds latency or produces unstable centroids.

Introduce hardware-in-the-loop validation. Feed recorded camera frames into the detector and estimator, command a representative gimbal or gimbal emulator, and compare predicted and measured response. Calibrate camera projection, latency, actuator saturation, and disturbance spectra against hardware data.

If the project evolves into a distributed multi-terminal simulator, move persistence to a service database, add experiment orchestration, and model explicit links between satellites, cameras, beacons, and tracking sessions. The current entity registry is a useful starting point for that evolution.

11. Overall Assessment and Conclusion

ASTERIA is a coherent and technically meaningful implementation of an AI-assisted FSOC coarse-alignment simulator. Its value comes from integrating the full feedback loop: physically motivated target and camera models, rendered observations, image-based detection, state estimation, bounded control, lock/loss state handling, telemetry, persistence, visualization, and reporting. The system is not merely a UI mockup and not merely a collection of disconnected algorithms.

The reviewed main branch is buildable and testable. The Python validation suite passed 39 tests. The frontend, backend, and Electron builds succeeded. The source contains several good engineering decisions: explicit coordinate conventions, finite-value sanitization, detector association memory, Kalman uncertainty, rate-limited PID output, SQLite writes off the event loop, and separate handling for synthetic ground truth versus external-video limitations.

The main qualification is evidentiary. The repository demonstrates a strong simulator and a credible baseline tracker, but it does not yet prove real-world optical tracking performance. The YOLO path is not an evaluated model, documented runtime measurements were not reproduced during this review, external-video ground truth is unavailable, and broad integration/load/hardware validation is absent. Those are normal next-stage gaps, not reasons to discard the architecture.

For a technical submission, ASTERIA should be presented as a **closed-loop FSOC PAT simulation and evaluation platform with classical computer vision, model-based state estimation, PID control, optional generative-AI assistance, and a YOLO integration path**. That wording is accurate, technically defensible, and aligned with the code actually present in the main branch.

References

Reproducibility note. The verification results in this report were obtained from the reviewed checkout at commit 1a53503. The Python command was `python3 -m pytest tests -q`; the build commands were `npm run build:frontend`, `npm --prefix backend run build`, and `npm run build:electron`.